

Publishing an AMI on AWS Marketplace: Process, Pricing Models & Considerations

Overview

AWS Marketplace enables you to **publish custom Amazon Machine Images (AMIs)** as commercial products that AWS customers can deploy with one click. To list your two AMIs (the man-in-the-middle proxy appliance and the caching service) on AWS Marketplace, you'll need to follow AWS's seller onboarding process and meet certain technical and documentation requirements. Below we outline the **step-by-step process** to go from an internal AMI to a public Marketplace listing, and discuss the **commercial models** (pricing and licensing options) AWS provides. We also highlight any **limitations/dependencies** and special considerations for your two use cases.

Prerequisites and AMI Preparation

Before publishing, ensure you have satisfied the prerequisites and prepared your AMIs according to AWS Marketplace guidelines:

- **AWS Marketplace Seller Registration:** You must **register as an AWS Marketplace seller** (using your AWS account). This involves providing business details and tax forms (W-9 for US or W-8 for non-US sellers) ¹. Registration is required to gain access to the AWS Marketplace Management Portal where you create listings.
- **Base AMI and Region:** Have your software installed and configured on an EC2 instance, and create an AMI from it. Typically, the AMI should be in the **US East (N. Virginia)** region (`us-east-1`), as AWS often uses that region for initial product setup ². (AWS will replicate the AMI to other regions you choose during listing.) Note the AMI ID.
- **Operating System and Credentials:** Use a supported OS (e.g., Amazon Linux, Ubuntu, Windows Server) and **do not include hard-coded credentials** or trial licenses that expire. The AMI should allow login via standard methods (for Linux, SSH with key using the default user like `ec2-user` or `ubuntu`; for Windows, RDP). For Linux-based products, ensure the SSH service on port 22 is enabled (for AWS's scanning) ³. For Windows, RDP on port 3389 should be accessible. Disable any unnecessary open ports or insecure defaults.
- **Security Updates and Best Practices:** Apply the latest security patches and follow AWS's **AMI best practices** (no malware, no plaintext passwords, etc.). AWS will **scan your AMI for security vulnerabilities** during the review process ⁴, so it should pass basic security and stability checks.
- **IAM Role for AMI Scanning:** Create an IAM role to allow AWS Marketplace to access and copy your AMI for scanning. AWS's process requires you to **grant AWS Marketplace permission to clone the AMI into their account** for vulnerability scanning ⁴. In the IAM console, create a role with the **"AWSMarketplace - AMI Assets Ingestion"** trusted entity and attach the `AWSMarketplaceAmiIngestion` policy ⁵ ⁶. You will provide this role's ARN in the listing steps so AWS can assume the role to copy/ingest your AMI. (This replaces the old method of manually sharing the AMI with a marketplace account.)
- **Third-Party or Open-Source Components:** If your AMI includes open-source or third-party software (for example, if the proxy is based on an open-source MITM proxy project), be mindful

of **AWS's repackaging requirements**. You must **add value** beyond the base software (e.g. additional features, configurations, or support), and make that clear in your product name/description. The product title should indicate the value added (e.g. *"CompanyName Proxy Server with Enhanced Caching"* rather than just the base project name) ⁷. If you charge for the product, the **short description must explicitly state that charges apply** and what they are for (for example: *"This product has charges associated with it for advanced caching and support"*) ⁸. In the long description, AWS requires a statement like *"This is a repackaged open-source software product wherein additional charges apply for [your value-add, e.g. support or extra features]"* ⁹. These disclosures ensure transparency when monetizing open-source software. Also, make sure you have rights to include any third-party components and adhere to their licenses.

Step-by-Step Process to Publish an AMI on AWS Marketplace

1. **Build and Test Your AMI:** (*Pre-step already done.*) You've indicated you have a build pipeline that configures an EC2 instance and creates (burns) an AMI. Before publishing, thoroughly **test your AMI** by launching it as a normal EC2 instance. Ensure your proxy AMI successfully starts the proxy service with the intended configuration, and your caching service AMI starts the caching service and can connect to the storage backend (file system or cloud storage) as expected. Verify that all required network ports are accessible and any configuration files or environment variables are properly set by default. This testing will reduce issues for Marketplace customers.
2. **Register as a Marketplace Seller:** Log in to the [AWS Marketplace Management Portal](#) using your AWS account. If you haven't already, complete the **seller registration** process (organization info, banking and tax info, etc.). Once registered, you can create Marketplace products. *Note:* AWS requires a valid bank account for disbursing revenue and may take a day or two to approve your seller account.
3. **Start a New Server Product Listing:** In the Marketplace Management Portal, navigate to the **Products** tab and choose **"Server"** (since an AMI is a server product). Click "Create product" and select **Amazon Machine Image (AMI)** as the product type ¹⁰. You'll be guided through a multi-step form for product details, AMI details, pricing, etc. (For more complex offerings you could use a Product Load Form Excel, but a simple single-AMI product can be done in the web portal UI.)
4. **Product Information & Branding:** Provide the basic information for your software:
 5. **Product Name:** A clear name for your AMI product. If repackaging existing software, include your company/brand and the added value in the name (per the guidelines). For example, *"MyCompany Secure MITM Proxy"* or *"MyCompany High-Speed Cache Server."*
 6. **Categories and Keywords:** Select relevant categories (e.g. *Security* or *Developer Tools* for the proxy; *Caching* or *Storage* for the caching service) and add keywords so users can find your product.
 7. **Short Description:** A one-liner (up to ~200 characters) shown in search results. This **must mention any charges** if it's a paid product – for example: *"This product has charges associated with it for enterprise support and maintenance"* ⁸. If the AMI is free or BYOL, you can simply summarize its function here without a charges disclaimer.
 8. **Long Description:** A detailed description of the product's features, use cases, and technical details. This can be a few paragraphs or bullet points. Explain what the AMI does (e.g. *"a man-in-the-middle proxy server that intercepts and logs HTTP(S) traffic for testing purposes"* or *"a caching service that speeds up read/write by caching data on disk or Amazon S3"*). Highlight the benefits and any unique features. If you are charging for a repackaged open-source tool, start

the long description with the required disclosure (e.g. *“This is a repackaged open source software product wherein additional charges apply for the added caching layer and support”*) ⁹ .

9. **Product Logo:** Upload an image for your product logo (PNG with transparent or white background). AWS requires logos to be 120–640 px in at least one dimension, with 1:1 or 2:1 aspect ratio ¹¹ . Ensure it’s a professional-looking icon relevant to your product.
10. **Support Information:** Provide details on how customers can get support. This might be an email address, support website, or a link to documentation/FAQ. If you offer paid support or an SLA, describe it here or in the long description.
11. **Configure AMI Details:** In the listing form, you will specify the details of the actual AMI:
 12. **AMI ID and Region:** Enter the AMI ID from your account (the one in us-east-1 that you prepared). The portal will validate that the AMI exists and is accessible.
 13. **IAM Role ARN:** Provide the ARN of the IAM role you created for AMI ingestion (from the **Prerequisites** step). This role allows AWS to copy and scan your AMI ⁴ .
 14. **Operating System & Version:** Select the OS (e.g. Amazon Linux 2, Ubuntu 20.04, etc.) and version for your AMI so customers know the base environment ¹² .
 15. **Default OS User:** Specify the default login username for the AMI (e.g. `ec2-user` for Amazon Linux, `ubuntu` for Ubuntu, etc.) ³ . This helps users know how to connect.
 16. **Network Ports / Security Group:** List the network ports your application uses and recommend a security group. AWS lets you define up to 12 **security group rules** for the product, which will be suggested to customers on launch ¹³ . For example, you’ll include an SSH rule (`tcp, 22, 22, 0.0.0.0/0/0`) for Linux or RDP (`tcp, 3389, 3389, . . .`) for Windows by default ¹³ . If your **MITM proxy** listens on a specific port (say 8080 or 8888), or your **caching service** uses a port (e.g. 11211 for Memcached, or an HTTP port), add those as well so they’re open to the user’s network. Only include necessary ports, and use restricted source ranges if appropriate (or let the customer adjust). Defining these helps AWS perform the vulnerability scan on the correct port and auto-creates a security group for users ¹⁴ .
 17. **Architecture & Instance Type:** Indicate if the AMI is x86_64 or ARM64, and recommend an instance type. For example, if the proxy or cache is lightweight, you might recommend a smaller instance like `t3.medium`, but if it’s resource-intensive you might suggest something larger. This recommended type appears on the product page, and you can **enable which instance types are supported**. Ensure your software runs on all instance types you mark as supported.
 18. **Regions Availability:** Choose the AWS regions where you want the product to be available. You can generally select “all regions” (except GovCloud unless you specifically handle GovCloud requirements) so customers globally can launch it. AWS will copy your AMI to the selected regions during publishing. You can also exclude any countries if needed for compliance (the form has “Countries to exclude” – usually left blank unless you have export restrictions) ¹⁵ .
 19. **Usage Instructions & Documentation:** AWS requires you to provide **usage instructions** that guide the customer on how to launch and use your AMI ¹⁶ . This is critical, especially for appliances like yours:
 20. In the **Usage Instructions field**, write a step-by-step guide for getting started. Assume the user is **“interested but uninformed”** ¹⁷ – i.e., don’t assume they know how your proxy or cache works. For example, instructions might be: “Launch the instance and then access the web UI at `https://<instance-ip>:<port>`” or “SSH into the instance and run XYZ command to start caching.” Provide default credentials if applicable. (AWS suggests using the instance ID or

another unique value as a default password if your app requires login ¹⁸ . If your proxy uses a web interface, ensure you've documented how to log in or use it.)

21. Include any configuration that the user must do. For the caching service, if it can connect to S3 or another storage, instruct the user how to configure that (e.g. setting an environment variable or attaching an IAM role with S3 access to the instance). For the proxy, if the user needs to configure client devices to use it as an HTTP proxy, provide guidance.
22. If your product requires additional AWS resources or setup (for example, creating an S3 bucket for cached data), mention those steps here or in an attached document. In more complex cases, sellers sometimes provide a link to a detailed user guide or even a CloudFormation template to set up related resources, but at minimum, list out what the customer needs to do.
23. **Support and Contact Info:** Reiterate how the user can get help (e.g., "For any issues, contact support@mycompany.com or see our online docs at ...").
24. **EULA and Policies:** You will need to specify a license agreement. By default, AWS Marketplace listings use the standard AWS Marketplace EULA (which covers basic software terms), but you can provide a custom EULA URL if you have specific license terms. If you do that, host the PDF or text of your EULA on a public URL (like S3) and provide the link ¹⁹ . Also, include a refund policy statement (even if it's just "no refunds" or "contact us for support issues" for a free product, as AWS may require a note on refunds).
25. **Release Notes (Versioning):** When you first list the product, you might have a "Version 1.0" – you can add release notes for this version. If you update your AMI later, you'll create a new version in the portal and provide update notes. It's good to follow semantic versioning and clearly describe changes in each version's release notes (critical updates vs. minor fixes, etc.) ²⁰ ²¹ .
26. **Pricing and Commercial Model:** The next step is to configure how you want to **commercialize the AMI**. AWS Marketplace offers multiple pricing models out-of-the-box:
 27. **Free or Open-Source (No Charge):** You can list the AMI for free, meaning customers incur **no software charges** (they only pay AWS for the EC2 infrastructure). This is suitable if you just want broad adoption or plan to monetize outside the platform. Free listings have no usage fees and allow unlimited instances ²² .
 28. **Bring Your Own License (BYOL):** In a BYOL model, the AWS Marketplace **does not handle billing** – you distribute license keys to customers externally (perhaps via your sales team or a website). Users must **enter a license key in the software** to activate it, but from AWS's perspective the AMI is free ²³ . This model is useful if you already have a licensing system or want to vet customers yourself. *Example:* You could have the caching service AMI require a license file or key on first boot (if user has purchased one). Keep in mind AWS still requires the listing to go through compliance, and you must clearly tell users how to obtain a license.
 29. **Hourly Pay-As-You-Go:** This is the classic **on-demand pricing**. You set an **hourly rate** for running your software on a given instance. AWS will meter the instance-hours and bill accordingly. You can price each EC2 instance type differently (common to charge more for larger instances) or use one price across all. Hourly pricing is flexible for customers and easy to understand – e.g., "\$0.05 per hour on top of EC2 costs." You can also offer a **Free Trial** with hourly products: for example, one instance free for the first 7 days, after which normal hourly rates apply ²⁴ . (AWS limits trials to 5–30 days and one free instance at a time.) Free trials encourage users to test your proxy or cache without commitment.
 30. **Annual Subscription (Hourly with Annual):** To cater to customers who prefer upfront commitments (often for a discount), AWS allows an **annual pricing option** in combination with hourly. A customer can choose to purchase a **1-year license** for a specific instance size of your product, paying a lump sum. After paying, they can run that instance type "unlimited" for a year.

Any usage beyond the purchased units (e.g. running extra instances or different sizes) would fall back to hourly rates ²⁵. You, as the seller, can set the annual price and typically offer it at a discounted equivalent (e.g. pay 10 months' worth and get 12 months access). Annual subscriptions are handled entirely by AWS Marketplace (they charge upfront and remit to you after fees).

31. **Monthly Subscription:** Instead of charging per hour, you can charge a **fixed monthly fee** for the software ²⁶. The customer is billed this flat rate each month (prorated if they subscribe or cancel partway through a month) regardless of how much they use the AMI. This model makes sense if your value is not tied to hours run – for example, if you expect the instance to run 24/7 and want a simple billing or if you license per month of usage. You can also combine monthly and hourly (charging a smaller hourly fee in addition to the base monthly) ²⁷, though combining with annual or free trial has restrictions ²⁸ ²⁹.
32. **Usage-Based (Metered by Units):** AWS Marketplace supports **custom metering** for AMIs via the *Marketplace Metering Service*. This means you can charge based on metrics like the number of **users, amount of data processed, bandwidth, or hosts/nodes managed**, etc., instead of (or in addition to) charging per instance-hour ³⁰. AWS defines four broad usage categories (users, data, bandwidth, hosts) for metering; you can create up to 24 custom dimensions within these. For example, for the caching service, you might charge per GB of data cached, or for the proxy, per GB of traffic inspected. Each hour, your software would call an AWS API to **report usage** for the instance (including your product code and the usage amount) ³¹ ³². AWS aggregates these and bills the customer monthly for the usage. Setting this up requires more development work: you'd integrate the AWSMarketplaceMetering SDK into your proxy/caching software to meter and send usage data ³³ ³⁴. Unless your use case truly demands granular usage billing, you might stick to simpler models like hourly or monthly. Metering is powerful for SaaS-like pricing on AMIs but be prepared to implement and maintain the reporting logic.
33. **Contract Pricing (Upfront Deals):** AWS Marketplace also allows **contract-based selling** for AMIs ³⁵. This typically means the customer pays an upfront fee for a fixed term (for example, a 1-year contract for the software, sometimes with the right to run it on a certain number of instances). Contract pricing is often used in **Private Offers** – a scenario where you negotiate custom terms with a specific customer (e.g. a multi-year enterprise deal at a custom rate). In the public listing, you might not enable a generic contract option unless you have a standard package (for instance, “\$X for a 1-year license, unlimited usage on one instance” as a public offer). Many sellers stick to hourly/annual models for public listings and use private offers for custom deals. It's good to be aware that multi-year and flexible payment schedules are possible via private offers ³⁶ ³⁷, but those are handled outside the immediate public listing flow.

Choose the model that best fits your product's value proposition: For example, if the **MITM proxy AMI** is typically used temporarily in test environments, an hourly (on-demand) charge might be most suitable. If the **caching service** is something users will run continuously in production, a monthly or annual subscription could be attractive for its predictability. You could also start with a simple hourly pricing and later introduce more options. In the Marketplace portal, you will input the prices for each instance type (for hourly) or the flat monthly/annual rates as applicable ³⁸ ³⁹. AWS will calculate and show you the customer-facing pricing. Remember to also set **refund policies** if needed (for example, you might state that charges are non-refundable once incurred, except as required by AWS Marketplace terms).

1. **Finalize and Submit Listing:** After filling in all the above details (product info, AMI details, usage instructions, pricing), you will review and **submit the product for publication**. At this point, the status of your listing will become “**Submitted**” in the portal and it enters AWS's **review process** ⁴⁰. The AWS Marketplace team will now do several things behind the scenes:

2. **AMI Ingestion and Scanning:** AWS will **copy your AMI** into their own account and run an automated security scan ⁴⁰. The scan checks for issues like open ports that weren't declared, known vulnerabilities, malware, or other policy violations. Ensure that your AMI's software doesn't have critical CVEs unpatched, as severe findings could cause AWS to reject the listing or ask you to fix them. The scan also verifies that the login on the stated port works (e.g., that SSH is accessible on port 22 for Linux, or RDP on 3389 for Windows) and that there are no hardcoded credentials. This scanning process usually takes about **an hour** to complete ⁴¹ for each AMI version.
3. **Content Review:** The Marketplace team will review your product description, details, and ensure everything is compliant with their guidelines. If anything is missing or unclear (for example, if usage instructions are insufficient or a logo is improper), they will mark the request as **"Action Required"** and ask you to make corrections ⁴². You can then update the listing draft and resubmit.
4. **Approval and Preview:** Once your submission passes review, its status will change to **"Approved – Limited"** or **"Approval Required."** **AWS will provide a limited listing URL – a private preview link where you (and anyone you share the link with) can see exactly how your product listing will appear on the Marketplace** ⁴³ ⁴⁴. **At this stage, the listing is not yet public to everyone, but you can use the URL to do a final QA. Only your account and AWS internal accounts can find the product by searching at this point** ⁴⁵. **Review all details (pricing, descriptions, screenshots if any, etc.) on this preview. If everything looks good, you then approve** the listing for publishing.**
5. **Public Publishing:** After you approve the limited listing, AWS will flip the product to live **public** status. The status might show "Publishing Pending" briefly during propagation ⁴⁶. Then your AMI product becomes discoverable in the Marketplace catalog for all customers. AWS will automatically replicate the AMI to all the regions you selected and make it available for subscribers. Congrats – your AMI is now live on AWS Marketplace!
6. **Post-Publication Considerations:** Once live, monitor your listing. You can update it by adding new AMI versions (e.g. when you have an updated build) – these updates don't require full re-review of everything, just the new version and release notes. Customers who subscribed can be notified of new versions. If at any point you need to change pricing or other core settings, AWS might require a review (and note that **you cannot change from free to paid model later without special approval** ⁴⁷, so choose your model carefully up front). You also have access to a **Marketplace analytics dashboard** for your product (downloads, revenue, etc.) and can see subscriber counts. Be prepared to respond to customer support inquiries that come via AWS (Marketplace can forward questions to your support contact). Keeping your product updated and well-supported will lead to better reviews and traction.

Commercial Models and AWS Marketplace Pricing Options

AWS Marketplace's pricing structure for AMI products provides flexibility in how you monetize your software. Below is a **summary of the business model options** (many of which we already touched upon) and how AWS handles billing for each:

- **Free Products:** The AMI has no associated software charges. Customers pay \$0 to you; they only pay AWS for the EC2 instance run. Free AMIs are often used for open source software or community editions. *Example:* If you decided to offer the MITM proxy AMI as a free tool, users could deploy it without any Marketplace fees ²². (Note: Even free products must go through the same listing and scanning process for security/compliance.)

- **Bring Your Own License (BYOL):** You distribute licensing outside of AWS. The Marketplace listing will show as “Free” (since AWS isn’t billing the user), but the **software will require a license key/activation** that the user must obtain from you. AWS does not enforce the license – your AMI should include logic to check for a valid key. BYOL is useful if you have existing customers or want a relationship directly with users. However, it adds friction for new users (they must contact you or your website to get a key). AWS expects you to handle all payment for that license out-of-band. They simply provide the EC2 deployment vehicle ²³.
- **Paid AMI – Hourly Pricing:** This is the standard **pay-as-you-go** model. AWS **meters the instance hours** that your AMI runs and bills the customer accordingly. For example, if you set a price of \$0.10/hour for the caching service, and a customer runs it 10 hours in a month, AWS will add \$1.00 to that customer’s bill for your software (plus the EC2 costs). Hourly charges are **pro-rated to the hour** and will round up to full hours ⁴⁸. You can set different rates per instance type if, say, a `4xlarge` provides more value than a `small` instance. With hourly pricing, you can optionally enable:
 - *Free Trial:* Let the customer run **one instance for a limited time (5–30 days)** at no charge ²⁴. This encourages adoption – after the trial period or for additional instances, normal hourly billing applies.
 - *Hourly + Monthly Combination:* You can charge a smaller hourly rate in combination with a fixed monthly fee (the monthly is charged regardless of usage, the hourly on top) ²⁶. However, note that **monthly fees cannot be combined with free trials or annual options** ²⁸ ²⁹ – combination models can get complex, so AWS restricts certain mixes.
- **Annual and Multi-Year Commitments:** A **1-year subscription option** (annual pricing) can be layered on an hourly product ²⁵. The user has the choice: pay as you go hourly **or** purchase an annual license for a single instance (for a defined instance type). Annual purchases lock in the software fee for one instance for 12 months; any additional instances are still billed hourly. Many vendors price annual subscriptions at roughly 30–40% savings compared to 12 months of hourly, to incentivize upfront commitment. Additionally, through the **Private Offer** mechanism, you can create custom SKUs for 2-year or 3-year contracts, or even custom durations, often with volume discounts or payment installments ³⁶ ³⁷. These longer terms are usually negotiated with specific customers (not shown by default on the public listing). For a public listing, you can stick to offering a 1-year upfront option and handle multi-year deals privately if needed.
- **Fixed Monthly Subscription:** In this model, customers pay a **flat monthly price** for the software, independent of runtime hours ⁴⁹. AWS will bill the customer this fee when they subscribe (prorating for partial months). This is akin to a SaaS subscription but for an AMI. It can be suitable if your software’s value doesn’t scale with hours or if you want predictable revenue per deployment. For instance, if your caching service is typically run 24/7, charging \$X per month per instance might be simpler than hourly math. You could also use monthly fees to cover support or maintenance. Just remember, with a pure monthly model, a user could run multiple instances and still pay only one fee (AWS treats it as one subscription per account). To prevent abuse, some vendors in this model design their software to phone home or limit to one active instance per subscription, or they combine monthly with a small hourly fee for additional instances ²⁷.
- **Usage-Based Metering:** If neither instance-hours nor flat subscriptions reflect how you want to charge, AWS’s **flexible metering** lets you bill for actual usage metrics ³⁰. AWS currently supports four categories for metered units:

- **Users:** e.g. charge per user account created or active.
- **Data:** e.g. charge per GB of data processed, cached, or transferred.
- **Bandwidth:** e.g. per GB of data throughput.

- **Hosts:** e.g. charge per host/device managed by the software.

You define specific **dimensions** (up to 24) within these and set a price per unit for each. Your software then calls the AWS Marketplace Metering API to **report usage each hour** for each customer instance ³² ³⁴. AWS tallies the usage and bills the customer monthly. For example, you might charge \$0.01 per GB of cache data written. If a user's instance writes 500 GB to the cache in a month, AWS would bill \$5.00 for that usage. This model aligns price with value, but it requires that your application track usage accurately and handle cases like missed reports or service interruptions gracefully. Also, usage fees are always billed on a monthly cycle (even though usage is metered hourly behind the scenes) ⁵⁰. If your two products have clear measurable metrics (requests, data cached, etc.) and you want a usage-based revenue model, this is the path – albeit with more engineering effort.

- **One-Time & Contract Payments:** Under **contract pricing**, a customer pays an upfront fee for a defined term of use ³⁵. For example, you could sell a “1-year license” for \$X. AWS collects the payment and the customer then has access to the software for that period. This is typically presented as **annual or multi-year SKUs** on your listing or done via private offers. There isn't a concept of a perpetual one-time fee in AWS Marketplace (all contracts have a time limit). If you wanted a “lifetime license” you'd likely just do a multi-year (3-year max via the private offers, or auto-renewing annual). Contract options are useful if you want to package support or updates as part of a term license. Keep in mind, some customers prefer the pay-as-you-go model for flexibility, so offering both can cater to different needs.

AWS handles all billing and payments for the above models (except BYOL). Customers are charged through their AWS account, and AWS then **remits the revenue to you**, the seller, typically on a monthly basis after taking a marketplace commission (around 20% of the software charges, though AWS doesn't publicly state it in docs, it's commonly understood). The advantage is you don't have to build a billing system or chase payments – AWS does it and provides you a detailed report of sales. You can also leverage **AWS's global marketplace** reach – any AWS customer in an allowed region can buy your software with a few clicks, and their existing AWS payment method is used.

When configuring your product's pricing in the portal, you'll select the model (e.g. *Hourly*, *Hourly with Annual*, *Monthly*, *Usage-based*, etc.) and fill in the pricing details accordingly ³⁸ ³⁹. The portal will guide you (for instance, if you pick “Hourly with Annual”, it will show fields for hourly prices per instance and annual prices; if you pick “Usage”, it will prompt you to define the usage dimensions and rates, etc.). Make sure to **double-check the final prices** in the preview – the listing will show the effective hourly rate or annual rate to customers. Also consider if you want to offer a **free tier or trial** to attract users; with AWS Marketplace, this would typically mean enabling a free trial period or listing a separate free edition of your AMI with limited features.

Additional Considerations for Your Use Cases

1. Man-in-the-Middle Proxy AMI: This appliance sounds like a network/security tool that intercepts and possibly logs or manipulates traffic (perhaps using something like mitmproxy or a custom proxy API integration). Here are a few specific things to consider when listing it:

- **Category & Compliance:** It could be categorized under **Security** or **Developer Tools** (if it's for testing). Make sure your product doesn't violate any AWS policies – for instance, tools that intercept traffic are generally allowed for legitimate use (like testing your own apps), but obviously customers must use

them ethically. It's good to include a note in your description about the intended use (e.g. for security testing, QA, caching of web content, etc.).

- **Configuration & Usage:** Document how to **launch and configure the proxy**. If it requires setting up SSL certificates (common for MITM proxies to generate a root cert for intercepting HTTPS), provide instructions on how the user can retrieve or trust that certificate. If the proxy has a web UI or command-line interface, explain how to access it. For example: "After launch, SSH to the instance and run `proxysql setup` to get the administration URL and credentials," or "Open a browser to `http://<instance-ip>:8080` to access the proxy dashboard." The more straightforward you make it, the better the reviews you'll get.

- **Security Group:** Ensure the security group rule for the proxy's listening port is included (e.g. TCP 8080) so that when customers deploy via the Marketplace UI, that port can be opened. If the proxy is meant to be private, you might instruct customers to restrict that port to their IP or VPC. Also consider that the proxy will need egress internet access (but that's usually allowed by default in AWS networking).

- **Instance Size:** A proxy might need a decent CPU and memory if it's handling a lot of traffic. Recommend an instance type that gives good performance (maybe C5 or M5 family). Conversely, if this is for testing, users might run it on smaller instances just for occasional use – so set your pricing to be reasonable even on small instance types to encourage usage.

- **Logging & Storage:** If the proxy caches or logs traffic, mention how/where it stores data. If it's writing to the instance disk, customers might want to know to use an appropriately sized EBS volume. If it can upload logs to S3 (just an example), mention that and any required config.

- **Value-Add:** Emphasize what **your custom proxy AMI adds** beyond someone just installing a proxy themselves. Maybe you've integrated it with an API or have pre-configured it for performance. These points should appear in the description to justify any pricing. For instance, "This AMI provides a ready-to-run MITM proxy with cloud-friendly caching and an API integration for dynamic rule updates, saving you time and effort." If you offer support or regular updates as part of the package, state that too.

2. Caching Service AMI: This sounds like a caching layer that can use local storage or cloud storage (possibly a custom cache or something like Redis/Memcached + an S3 backup?). Consider the following:

- **Intended Use & Category:** Possibly falls under **Databases** or **Storage** category if it's a general-purpose cache, or **Application Infrastructure**. Clarify the use case: e.g., "A caching service that speeds up applications by storing frequently accessed data in memory and syncing to S3 for durability." If it's specific (like a disk cache for web content), mention that.

- **Configuration:** Describe how to use the cache. For example: does it expose a Memcached/Redis protocol on a port that apps connect to? Or is it a filesystem cache that requires mounting it somehow? If the caching can be toggled between filesystem vs cloud, instruct how to choose the mode (perhaps via an environment variable or config file). If the user needs to provide an S3 bucket name or credentials for cloud caching, instruct them to either supply AWS credentials or (better) attach an IAM role with the necessary S3 permissions to the EC2 instance when launching. This can be part of the usage instructions. Remember to **never hardcode your own AWS keys** in the AMI – always let the customer use their own resources.

- **Security & Access:** Caches often listen on a port (e.g. Redis default 6379, Memcached 11211, or maybe HTTP if it's an HTTP cache). Ensure the default configuration is **secure** – e.g., bound to localhost or the VPC only, or requires authentication if applicable. You don't want customers inadvertently launching an open Memcached to the world. Recommend in the instructions that they should limit access to the cache port to their application's security group or use AWS security group rules accordingly. You might include in the **security group recommendations** that the cache port is open only to the user's VPC CIDR (Marketplace allows CIDR specification in the SG rules you provide ¹³). If your cache has a web console, similarly secure it.

- **Performance Considerations:** Provide guidance on instance sizing and storage. For example, if it's disk-heavy caching, suggest using **instances with high-performance disks or attaching Provisioned IOPS volumes**. If it's memory heavy (in-memory cache), suggest more RAM. This information can go in the

long description or usage notes, helping customers choose the right instance type.

- *Persistence and Backup*: If the cache can persist data (especially using S3 or EBS), outline how that works. For instance: *“The cache automatically backs up files to the configured S3 bucket every hour”* or *“Data is persisted on the instance’s EBS volume at `/var/cache`; consider using an EC2 instance with an EBS volume that has sufficient throughput.”* Also instruct how to **backup or recover** if needed (since cache might hold important data in some cases). This aligns with AWS’s guideline to explain backup/recovery steps for any proprietary data store ⁵¹ ⁵² .

- *Testing and Demo*: Customers may want to validate the cache in a lower environment. If feasible, you could offer a **free tier** (maybe a t2.micro free AMI if it’s open source) or at least a free trial on the paid product so they can try it out.

Dependencies: Both of your AMIs can integrate with other AWS services (the proxy might not, but the caching one with S3 does). AWS Marketplace doesn’t forbid this – just make sure to **clearly state any external requirements**. For example, *“This caching service can optionally use Amazon S3 for storage – the user must configure an S3 bucket and IAM role (not provided) to enable that feature.”* Also, **ensure your AMI doesn’t automatically make unexpected external calls** on its own (aside from intended ones like calling S3 if configured). During scanning, AWS might flag unusual outbound network behavior. But if it’s just standard operations, it should be fine.

Another limitation to note: AWS Marketplace **does not allow multi-AMI bundles in a single product listing** unless you use a CloudFormation template. Each AMI product listing is basically one AMI (one machine image) per fulfillment option. If your solution had multiple components (say a two-tier system), you’d use a CloudFormation-based listing. In your case, you have two separate AMIs that serve different purposes – these should be **listed as two separate products** on the marketplace (each with its own pricing, description, etc.). You can cross-reference them (for example, in the caching service listing, say “Pairs well with our Proxy AMI available [here]”), but they will be subscribed to independently by users.

Documentation & Support: As part of your listing, consider preparing a PDF or online documentation that you can link to. While the usage instructions field covers the basics, detailed user guides (installation, troubleshooting, FAQs) can be hosted on your website or in an S3 bucket and referenced. AWS Marketplace doesn’t host large docs for you, but you can include hyperlinks. Also decide how you will handle support queries – e.g., if a user has an issue deploying the proxy, who do they contact and what is the response time? If you’re charging for the product (especially if one of the value propositions is that it’s supported software), you should at least provide an email or ticketing system for customers.

Finally, keep in mind any **legal/licensing limitations**. For example, if your proxy includes any encryption libraries or does cryptography, ensure compliance with export laws (AWS will ask if your product includes any crypto that might be subject to export control – usually open-source crypto in common libraries is fine, but you have to confirm compliance with AWS standards during listing). And if your caching service includes any third-party licensed components, ensure those licenses allow redistribution or are properly included.

Conclusion

Deploying your custom AMIs to AWS Marketplace is a multi-step process, but it enables you to **commercialize your solutions globally** with minimal friction. You’ll start by registering as a seller and meticulously preparing your AMIs (security, documentation, pricing). AWS provides a variety of **business models**, from pay-as-you-go hourly billing to subscriptions and custom usage-based pricing, so you can choose what fits your products and customers (e.g. hourly for the on-demand proxy, subscription for the always-on cache, or any combination). Throughout the process, adhere to AWS’s

guidelines on transparency and security – clearly state what you’re offering and any charges ⁸, and ensure your AMIs are robust and safe, as AWS will scan them before allowing publication ⁴.

By focusing on your two use cases in the listing content (explaining the proxy and caching service benefits, usage, and requirements) and leveraging AWS Marketplace’s built-in billing, you can reach **all typical AWS audiences** (developers, IT admins, etc. in need of your solutions) via a public listing. Once published, your products will be available for any AWS customer to discover and deploy with a few clicks, and AWS will handle the **infrastructure provisioning, billing, and distribution**. This leaves you to concentrate on what you do best: improving your proxy and caching services, and supporting your users. Good luck with your AWS Marketplace launch!

Sources: AWS Marketplace Seller Guide (AMI product pricing models) ²² ³⁰, AWS Marketplace product submission documentation ³⁸ ⁴, AWS guidelines for product descriptions and instructions ⁸ ¹⁸, and relevant best-practice insights.

¹ ²² ²³ ²⁴ ²⁵ ²⁶ ²⁷ ²⁸ ²⁹ ³⁰ ³⁵ ³⁶ ³⁷ ⁴⁸ ⁴⁹ ⁵⁰ AMI product pricing for AWS Marketplace - AWS Marketplace

<https://docs.aws.amazon.com/marketplace/latest/userguide/pricing-ami-products.html>

² ³ ¹² ¹⁴ ²⁰ ²¹ ³¹ ³² ³³ ³⁴ ⁴¹ ⁴⁵ Place free AMI product on the AWS Marketplace | tecRacer Amazon AWS Blog

<https://www.tecracer.com/blog/2023/01/place-free-ami-product-on-the-aws-marketplace.html>

⁴ ⁵ ⁶ Giving AWS Marketplace access to your AMI - AWS Marketplace

<https://docs.aws.amazon.com/marketplace/latest/userguide/single-ami-marketplace-ami-access.html>

⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹³ ¹⁵ ¹⁹ ³⁸ ³⁹ ⁴⁰ ⁴² ⁴³ ⁴⁴ ⁴⁶ ⁴⁷ Submitting your product for publication on AWS Marketplace - AWS Marketplace

<https://docs.aws.amazon.com/marketplace/latest/userguide/product-submission.html>

¹⁶ ¹⁷ ¹⁸ ⁵¹ ⁵² Creating AMI and container product usage instructions for AWS Marketplace - AWS Marketplace

<https://docs.aws.amazon.com/marketplace/latest/userguide/ami-container-product-usage-instructions.html>